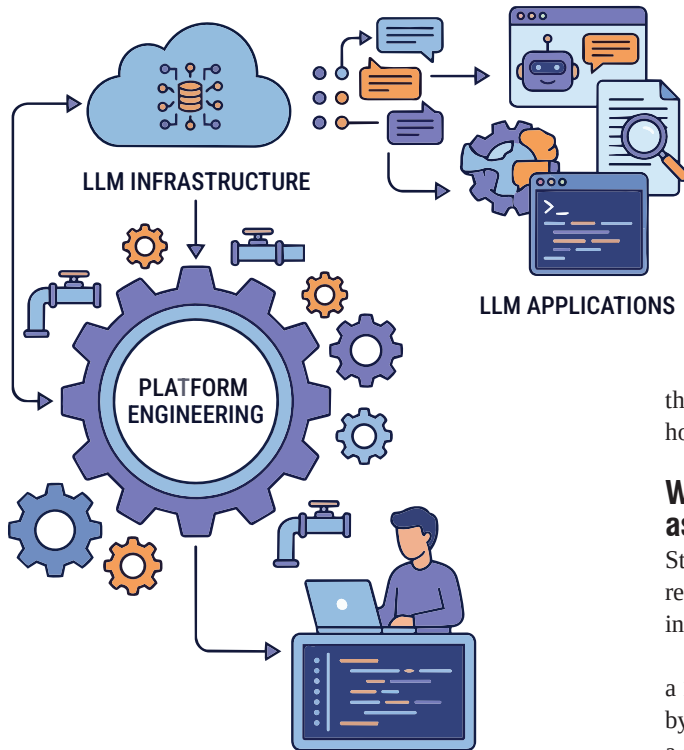


A Peek at Platform Engineering for LLM Applications



Glitches in an LLM service do not necessarily show up because of a poorly engineered platform. Even platforms built well can falter because LLM workloads violate reliable assumptions.

the patterns hold whether you self-host models or call a hosted API.

Why do LLM workloads break your platform assumptions?

Start with the request. In a conventional service, one request costs roughly what the last one costs, replicas are interchangeable, and adding capacity is a scaling decision.

An LLM request breaks all three. A 200 token prompt and a 120,000 token prompt travel the same code path and differ by orders of magnitude in compute. A replica already holding a conversation's key value cache answers far faster than an identical replica that does not. And the instance you want during a spike may not exist, because accelerator capacity is contracted rather than elastic.

A platform team that has spent five years perfecting Kubernetes golden paths, Terraform modules, and GitOps pipelines can still get blindsided by its first production LLM service. The pipelines work. The clusters are healthy. And the p95 latency chart looks fine while users complain, the bill triples after a one-line prompt edit, and a security review asks a question nobody can answer: what can this agent do if someone hides an instruction inside a support ticket?

None of that happens because the platform was built badly. It happens because LLM workloads violate assumptions that were so reliable they stopped being visible. This article works through three of them:

- Observability for a system where latency has two phases and correctness is a distribution.
- Security for a system where input, retrieved data, and model output are all untrusted.
- Cost for a system that bills per token and behaves like the cost of goods sold.

The tools named are open source or open standards, and

Classic assumption	What holds for LLM workloads	Platform consequence
Requests cost about the same	Compute varies 100x by prompt and output length	Quotas must count tokens, not requests
Replicas are interchangeable	Each replica holds its own KV cache state	Load balancing must be cache aware
Latency is one number	Time to first token and inter-token delay differ	Two SLOs, measured separately
Correctness is binary	Output quality is a distribution	Testing becomes statistical evaluation
Input is data	Input is also instruction	The model is a confused deputy by design